

SIMATS ENGINEERING

ASSESSMENT TOOL 1: Experiments

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO3: *Implement cloud-based solutions using cloud storage, web APIs, and platform services for real-world applications. (BL3)*

Assessment Tool Weightage: 40%

Dave's Taxonomy: Precision (Level 3)
Question Count: 5

Total Marks: 40

Code of Conduct

I ___G.Jaahnavi(192525277)___ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: ___G.Jaahnavi_____

Assessment Tasks Experiments

1. Experiment 1: Create and document a cloud storage comparison between object, block, and file storage. Select the best storage model for backups, VM disks, and shared enterprise documents.
2. Experiment 2: Develop a simple REST API workflow for a cloud-based student portal. Show request, response, authentication, and deployment flow.
3. Experiment 3: Design a serverless event flow for image upload processing using object storage, function-as-a-service, and notification service.
4. Experiment 4: Compare edge computing and fog computing for a smart traffic management system. Include architecture, latency considerations, and deployment location.
5. Experiment 5: Demonstrate a cloud application deployment workflow showing client access, browser interaction, web API call, storage access, and monitoring.

For each experiment, include objective, architecture sketch, procedure, expected output, and conclusion.

Experiments Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Experimental Design	25%	Design is systematic and technically strong	Mostly correct design	Basic design with gaps	Poorly designed activity

Use of Tools / Platforms	20%	Appropriate and effective use of cloud tools	Good tool usage with minor issues	Limited tool usage	Incorrect or missing tool usage
Application of Concepts	20%	Concepts are applied accurately to practical tasks	Mostly accurate application	Partial application	Weak application
Analysis and Output	20%	Strong analysis and meaningful outcome interpretation	Good analysis with minor gaps	Basic analysis	Little or no analysis
Documentation	15%	Clear, structured, and complete documentation	Mostly complete documentation	Partially complete	Poor documentation

Experiment 1: Cloud Storage Comparison

1. Objective

To study and compare the three major cloud storage models:

- Object storage
- Block storage
- File storage

The experiment also aims to select the most suitable storage model for:

- Backups
- Virtual machine (VM) disks
- Shared enterprise documents

2. Introduction

Cloud storage allows organizations to store data on remote servers managed by cloud service providers. Instead of maintaining physical storage devices, users can access storage through the network.

The three commonly used storage models are:

Object Storage

Object storage stores data as individual objects. Each object contains the actual data, metadata, and a unique identifier.

Examples: Amazon S3, Azure Blob Storage, Google Cloud Storage.

Suitable for:

- Backup files
- Images and videos
- Documents
- Logs
- Large unstructured datasets

Block Storage

Block storage divides data into fixed-size blocks and stores them separately. It behaves similar to a physical hard disk and is commonly attached to virtual machines.

Examples: Amazon EBS, Azure Managed Disks.

Suitable for:

- VM operating-system disks
- Databases
- Applications requiring low-latency storage

File Storage

File storage organizes data using files and folders. Multiple users or applications can access the same file system.

Examples: Amazon EFS, Azure Files.

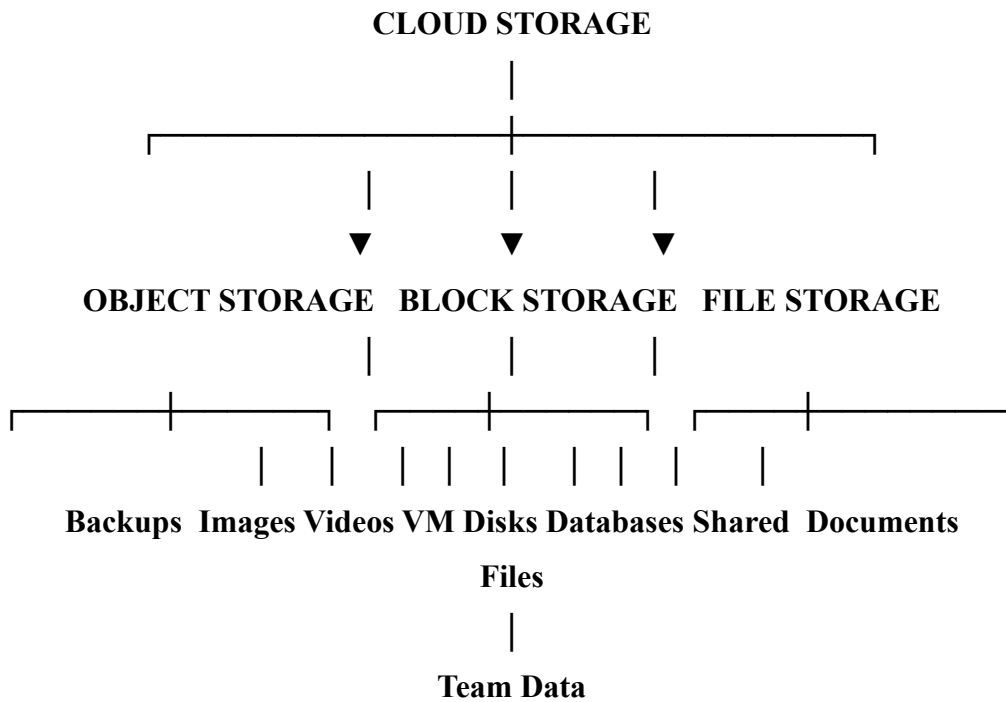
Suitable for:

- Shared documents
- Enterprise file systems
- Collaborative applications

3.Comparsion

Feature	Object Storage	Block Storage	File Storage
Data organization	Objects	Blocks	Files/folders
Access method	API/HTTP	Attached disk	File system
Scalability	Very high	High	High
Best for	Backups, media	VM disks, databases	Shared documents
Sharing	API based	Usually attached to VM	Multiple users
Performance	Good	Very high	Good
Example	Amazon S3	Amazon EBS	Amazon EFS

4. Architecture Sketch



5. Procedure

1. Identify the storage requirements of the application.
2. List the three storage models.
3. Study their architecture and access mechanisms.
4. Compare scalability, performance, accessibility, and use cases.
5. Map each requirement to the most suitable storage model.
6. Select the appropriate model based on the application requirement.
7. Document the comparison in a table.
8. Analyze the advantages and limitations of each model.

6. Storage Selection

A. Backups → Object Storage

Object storage is the best choice for backups because it provides highly scalable storage for large quantities of data.

Reasons:

- Supports large amounts of data.
- Suitable for unstructured backup files.
- Provides high durability.
- Can be accessed through APIs.

- Supports lifecycle and archival strategies.

B. VM Disks → Block Storage

Block storage is appropriate for virtual machine disks.

Reasons:

- Provides disk-like access.
- Offers low-latency operations.
- Suitable for operating systems and databases.
- Can be attached to virtual machines.
- Provides predictable storage performance.

C. Shared Enterprise Documents → File Storage

File storage is suitable for enterprise documents because users can access files using folders and file paths.

Reasons:

- Supports shared access.
- Provides familiar file/folder organization.
- Suitable for collaboration.
- Multiple applications can access shared files.

7. Expected Output

The final comparison should show:

Backup Documents → Object Storage

Virtual Machine Disk → Block Storage

Enterprise Documents → File Storage

8. Analysis

No single storage model is suitable for every application. Object storage provides excellent scalability for large unstructured data, block storage provides high-performance disk access, and file storage provides convenient shared file access.

9. Conclusion

The experiment demonstrates that storage selection depends on the application requirement. **Object storage is best for backups, block storage for VM disks, and file storage for shared enterprise documents.**

Experiment 2: REST API Workflow for a Cloud-Based Student Portal

1. Objective

To develop and document a simple REST API workflow for a cloud-based student portal & demonstrate:

- Request
- Response
- Authentication
- Deployment flow

2. Introduction

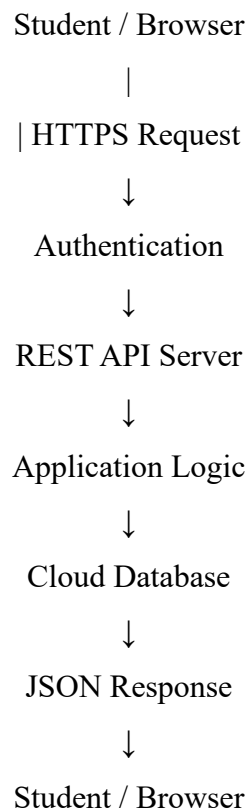
A REST API allows different software applications to communicate through HTTP.

For example, a student portal can use APIs to retrieve student information, attendance, marks, and course details.

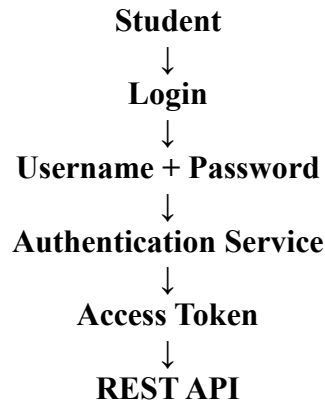
Common HTTP methods are:

- **GET** – Retrieve information
- **POST** – Create information
- **PUT** – Update information
- **DELETE** – Remove information

4. Architecture Sketch



6. Authentication



7. Response

The API can return a JSON response such as:

```
{
  "student_id": 101,
  "name": "Student",
  "department": "Computer Science",
  "year": 2
}
```

8. Procedure

1. Create the student portal REST API.
2. Implement user authentication.
3. Send API requests and receive responses.
4. Connect the API to a cloud database.
5. Deploy and test the API.

9. Expected Output

The deployed student portal should:

- Authenticate users.
- Accept API requests.
- Retrieve student information.
- Return JSON responses.
- Provide secure HTTPS communication.

10. Conclusion

The REST API workflow demonstrates how a cloud-based student portal can communicate with backend services. Authentication provides security, while REST APIs enable structured communication between the client, application server, and cloud database.

Experiment 3: Serverless Event Flow for Image Upload Processing

1. Objective

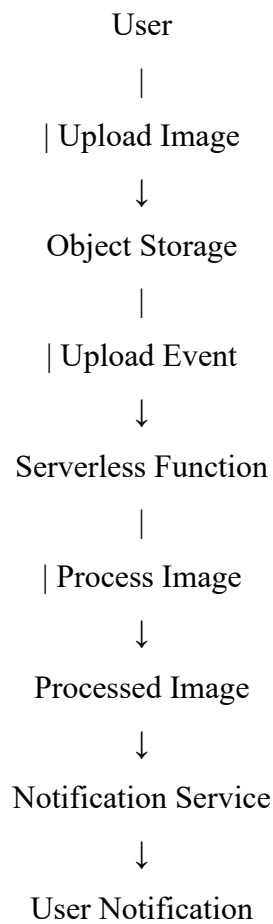
To design a serverless cloud workflow that automatically processes an image when it is uploaded to object storage using:

- Object storage
- Function-as-a-Service
- Notification service

2. Introduction

Serverless computing allows developers to execute application logic without managing servers directly.

3. Architecture Sketch



4. Components

Object Storage

Stores the uploaded image.

Examples include:

- Amazon S3
- Google Cloud Storage

Function-as-a-Service

Runs processing code automatically when an event occurs.

Examples:

- AWS Lambda
- Azure Functions

Notification Service

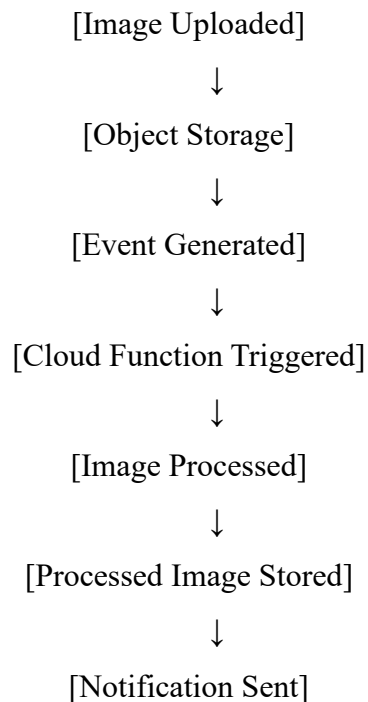
Sends a notification after processing.

It can notify users through an application, email, or messaging service depending on the cloud setup.

5. Procedure

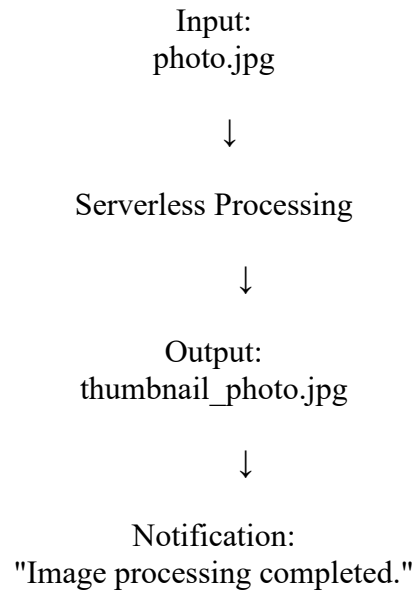
1. Create a cloud object-storage bucket.
2. Configure an upload location.
3. Upload an image.
4. Configure an event trigger for new image uploads.
5. Connect the event to a serverless function.
6. Write processing logic.
7. Execute the function automatically.
8. Store the processed image.
9. Trigger the notification service.
10. Verify the final output.

7.Event Flow



9. Expected Output

After uploading an image:



10. Analysis

The event-driven architecture reduces manual intervention. The image upload automatically triggers processing, making the system suitable for applications such as photo platforms, document processing, and media management.

11. Conclusion

The experiment demonstrates how object storage, serverless functions, and notification services can work together. An image upload becomes an event that automatically starts processing and sends the result notification.

Experiment 4: Edge vs Fog Computing

Objective

To compare edge and fog computing for smart traffic management.

2. Introduction

Smart traffic systems use cameras, sensors, traffic lights, and other devices to monitor road conditions.

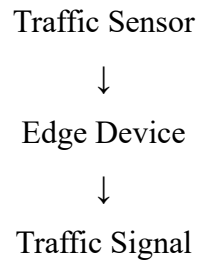
The system may need to process information quickly because traffic decisions can be time-sensitive.

Two suitable approaches are:

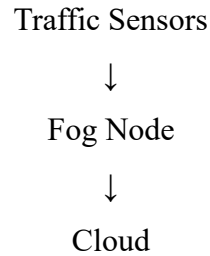
- Edge computing
- Fog computing

Architecture

Edge:



Fog:



Procedure

1. Collect traffic data from sensors.
2. Process data at the edge.
3. Process combined data through fog nodes.
4. Compare latency and deployment location.
5. Select the suitable approach.

5.Expected Outcomes

Immediate Traffic Decision → Edge

Regional Traffic Analysis → Fog

Long-Term Analytics → Cloud

6.Conclusion

Edge computing is highly suitable for immediate traffic decisions because processing occurs close to the sensors. Fog computing provides an additional intermediate layer for regional coordination and aggregation. A combined architecture can provide low latency while supporting large-scale traffic analytics.

Experiment 5: Cloud Application Deployment

1.Objective

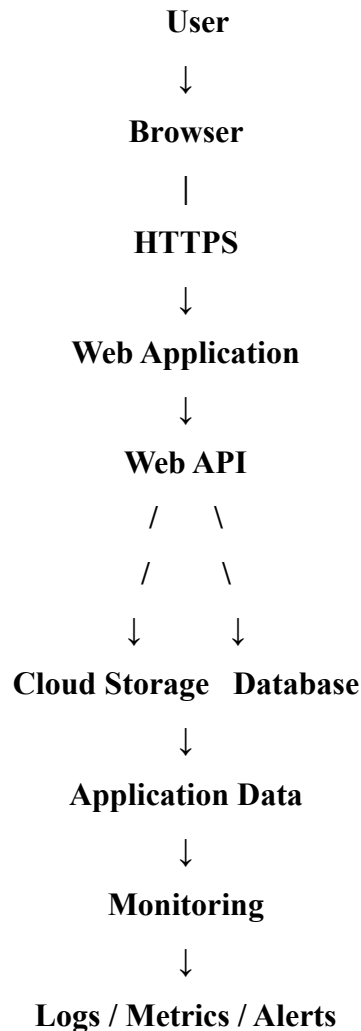
To demonstrate a cloud application deployment workflow involving browser access, APIs, storage, and monitoring.

2. Introduction

A cloud application typically consists of multiple components working together.

A user interacts with the application through a browser. The browser communicates with a web server or.

3.Architecture



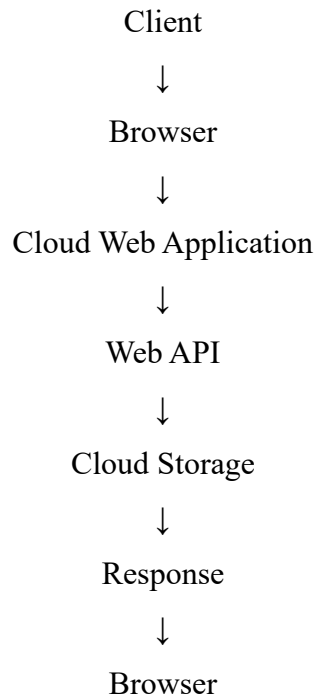
4.Procedure

1. Develop the cloud application.
2. Test the application.
3. Deploy it to the cloud.
4. Configure the web API.
5. Connect cloud storage.

6. Enable monitoring.
7. Access the application through a browser.

5.. Expected Output

The deployed application should demonstrate:



At the same time, monitoring should record application activity.

6. Analysis

A complete cloud deployment requires more than simply hosting a website. The application must connect the client, web interface, APIs, storage, and monitoring services.

Monitoring is especially important because it helps identify errors and performance problems after deployment.

7. Conclusion

The experiment demonstrates a complete cloud application workflow from client access to backend storage and monitoring. The architecture provides a structured approach for deploying and managing cloud-based applications.